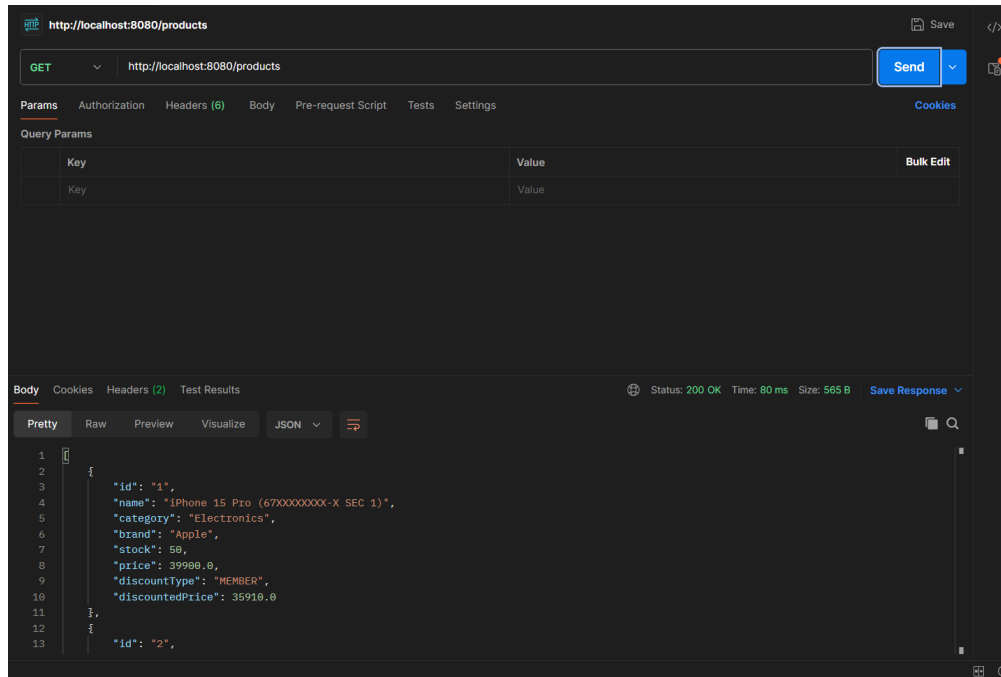
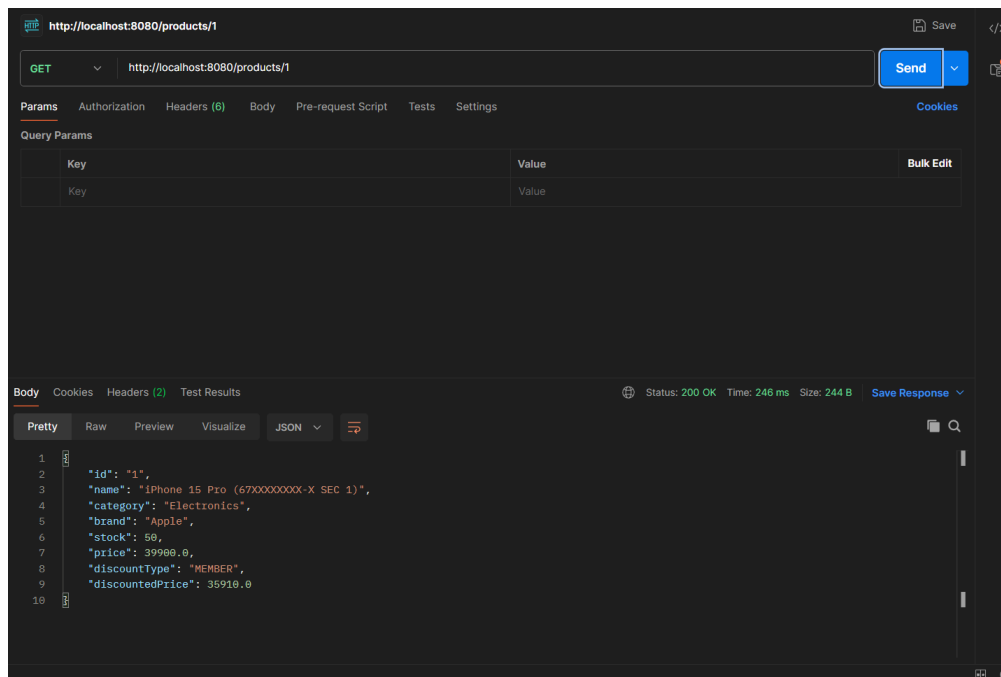


## ทดสอบ get method



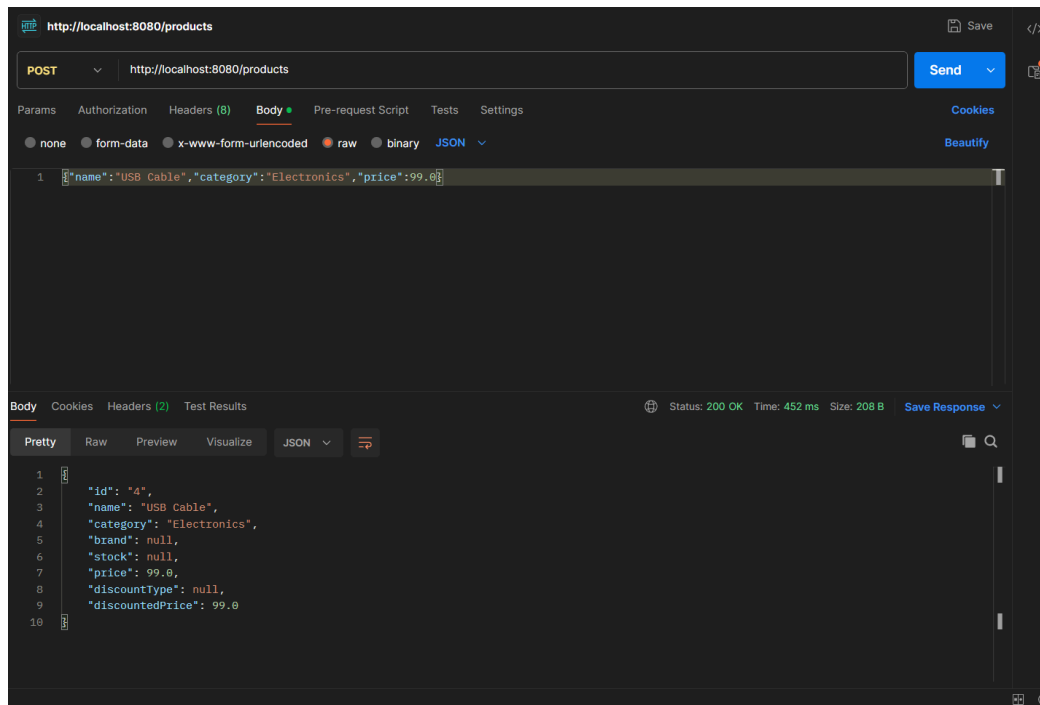
สามารถดึงข้อมูลมาใช้งานได้

## ทดสอบ GET /products/{id}



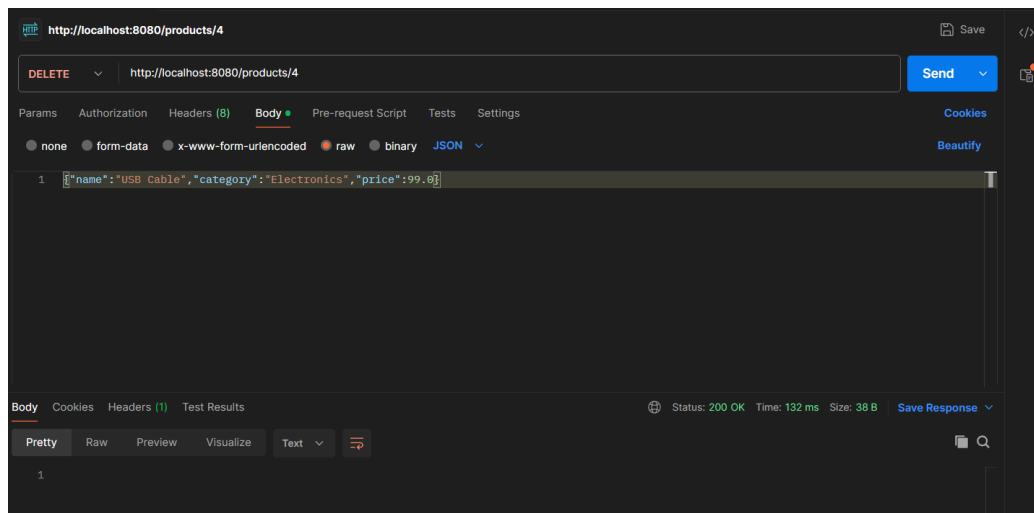
สามารถดึงข้อมูลตาม id ได้

## ทดสอบ POST /products



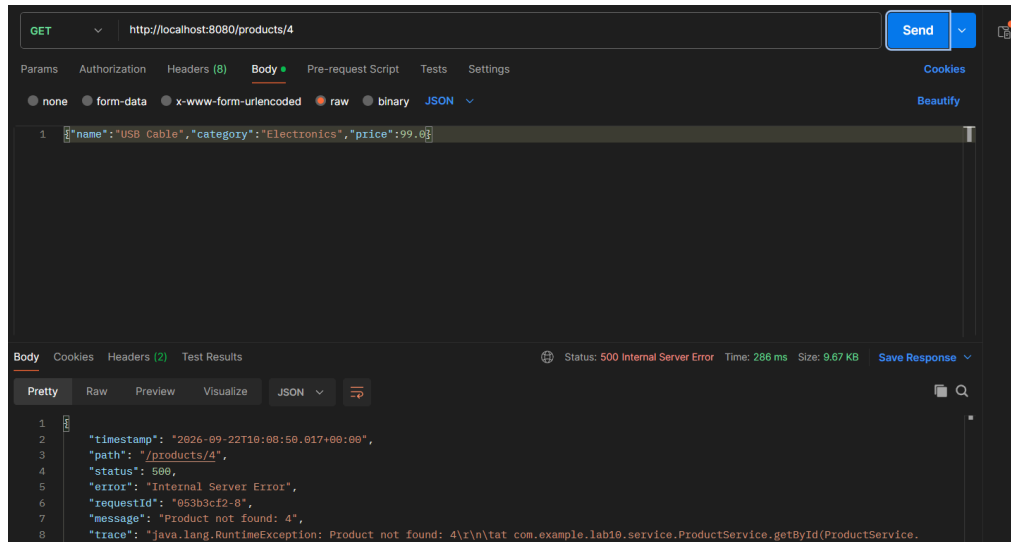
สามารถทำการเพิ่ม product ได้

## ทดสอบ DELETE /products/{id}



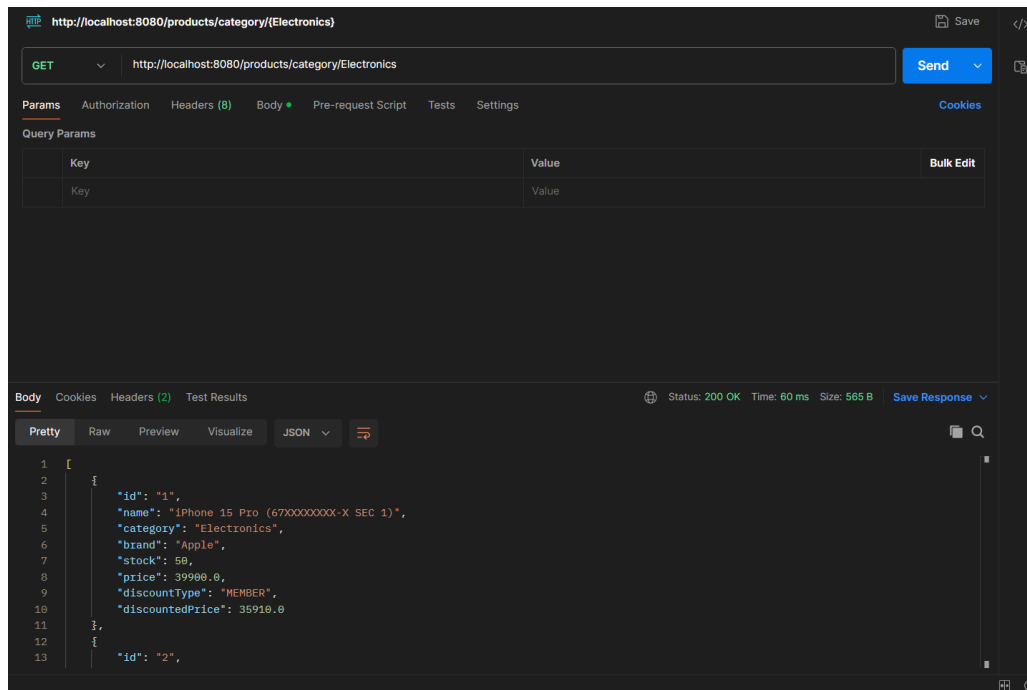
สามารถลบได้ตามต้องการ

## ทำการตรวจสอบว่าข้อมูลถูกต้องหรือไม่



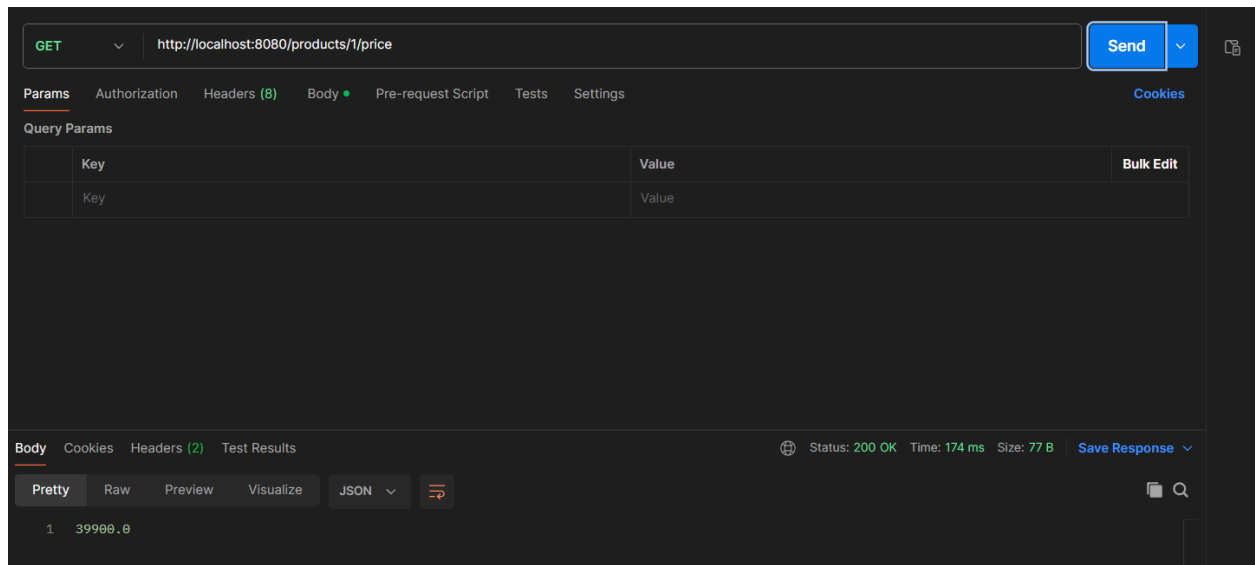
ไม่มี product id=4 อยู่แล้ว สามารถลบได้ตามต้องการ

## ทดสอบ GET /products/category/{category}



สามารถดึงข้อมูลตาม category ได้ตามต้องการ

## ทดสอบ GET /products/{id}/price



ได้รับราคาสินค้าที่ลดราคาแล้ว ตามที่ต้องการ

## ตอบคำถาม

1. Reactive Programming vs Blocking — ต่างกันอย่างไร  
ตอบ : แบบ Blocking คือ thread ต้องหยุดรอนกว่างาน I O จะเสร็จ ทำให้ต้องใช้ thread หนึ่งตัวต่อหนึ่ง request พอมี request เยอะก็เปลืองทรัพยากรมาก ส่วนแบบ Reactive คือ thread ไม่ต้องรอ ปล่อยงานไปทำเบื้องหลังแล้วไปทำงานอื่นก่อน พอผลลัพธ์พร้อมค่อยแจ็กกลับมาที่หลังผ่านสัญญาณสามแบบคือ onNext onError และ onComplete วิธีนี้ใช้ thread จำนวนน้อยรองรับ request จำนวนมากได้ เหมาะกับงานที่มีคนใช้พร้อมกันเยอะ
2. Mono vs Flux — ใช้กรณีไหน และทำไม  
ตอบ : Mono ใช้กับกรณีที่คืนค่าศูนย์หรือหนึ่งค่า เช่น findById save หรือ delete ส่วน Flux ใช้กับกรณีที่คืนค่าได้หลายค่า เช่น findAll หรือ findByCategory หลักการเลือกคือดูจากจำนวนผลลัพธ์ที่คาดว่าจะได้เป็นหลัก
3. WebClient — อธิบาย method chain .get(), .uri(), .retrieve(), .bodyToMono/Flux()  
ตอบ : get หรือ post คือการเลือกจะใช้ HTTP method แบบไหน  
uri คือการกำหนด path ปลายทางที่จะไปยัง  
retrieve คือจุดที่เริ่มยิง request ออกไปจริง และเตรียมอ่านผลลัพธ์ที่ตอบกลับมา  
bodyToMono หรือ bodyToFlux คือการแปลงข้อมูล JSON ที่ตอบกลับมาให้กลายเป็น object ของ Java แบบไม่ต้องรอบล็อก ทั้งหมดนี้จะไม่ทำงานจริงจนกว่าจะมีการ subscribe ซึ่ง Spring จะจัดการให้อัตโนมัติ

4. Code ของ Repository, Service, Controller พร้อม comment อธิบาย

ตอบ : Repository ทำหน้าที่เข้าถึงข้อมูลโดยตรง แล้วห่อผลลัพธ์ด้วย Mono หรือ Flux ตัวอย่าง เช่น findById จะเช็คว่ามีข้อมูลไหม ถ้าไม่มีจะกลายเป็นค่าว่างอัตโนมัติ ส่วน findByCategory จะเรียก findAll แล้วกรองด้วย filter ต่อบน stream เดิมเลยโดยไม่ต้องดึงข้อมูลออกมาก่อน

Service ทำหน้าที่ใส่ business logic เพิ่มเติมระหว่างทาง เช่น getById จะเรียก repository แล้ว ถ้าไม่พบข้อมูลจะโยน error ออกไปแทนที่จะคืนค่าว่างเฉยๆ

Controller ทำหน้าที่รับ HTTP request แล้วส่งต่อไปให้ Service ทำงาน เช่น getById จะรับค่า id จาก path แล้วเรียก service ไปคืนค่าเป็น Mono ของ Product กลับไปให้ผู้เรียก